Tries is a tree that stores strings. Maximum number of children of a node is equal to size of alphabet. Trie supports search, insert and delete operations in O(L) time where **L** is length of key.

Why Trie? :-

1. With Trie, we can insert and find strings in ***O(L)*** time where *L* represent the length of a single word. This is obviously faster than BST. This is also faster than Hashing because of the ways it is implemented. We do not need to compute any hash function. No collision handling is required (like we do in open addressing and separate chaining)
2. Another advantage of Trie is, we can easily print all words in alphabetical order which is not easily possible with hashing.
3. We can efficiently do prefix search (or auto-complete) with Trie.

Issues with Trie :-

The main disadvantage of tries is that they need a lot of memory for storing the strings. For each node we have too many node pointers(equal to number of characters of the alphabet), if space is concerned, then **Ternary Search Tree** can be preferred for dictionary implementations. In Ternary Search Tree, the time complexity of search operation is O(h) where h is the height of the tree. Ternary Search Trees also supports other operations supported by Trie like prefix search, alphabetical order printing, and nearest neighbor search. The final conclusion is regarding *tries data structure* is that they are faster but require *huge memory* for storing the strings.

Difference between BST and tries.

A **binary tree** or a **bst** is typically used to store numerical values. The time complexity in a bst is O(log(n)) for insertion, deletion and searching. Each node in a binary tree has at most 2 child nodes.

Trie is an ordered tree structure, which is used mostly for storing strings (like words in dictionary) in a *compact* way. In a trie, every node (except the root node) stores one character. By traversing up the trie from a leaf node to the root node, a string can be constructed. By traversing down the trie from the root node to a node n, a common prefix can be constructed for all the strings that can be constructed by traversing down all the descendant nodes (including the leaf nodes) of node n.

The following are the main advantages of tries over binary search trees (BSTs):

- Looking up keys is faster. Looking up a key of length *m* takes worst case O(*m*) time. A BST performs O(log(*n*)) comparisons of keys, where *n* is the number of elements in the tree, because lookups depend on the depth of the tree, which is logarithmic in the number of keys if the tree is balanced. Hence in the worst case, a BST takes O(*m* log *n*) time. Moreover, in the worst case log(*n*) will approach *m*. Also, the simple operations tries use during lookup, such as array indexing using a character, are fast on real machines.
- Tries are more space efficient when they contain a large number of short keys, because nodes are shared between keys with common initial subsequences.
- Tries facilitate longest-prefix matching, helping to find the key sharing the longest possible prefix of characters all unique.
- The number of internal nodes from root to leaf equals the length of the key. Balancing the tree is therefore no concern.